

svm_E1

May 14, 2025

```
[317]: from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
import numpy as np

def make_grid(X: np.ndarray, target_points: int = 10000) -> np.ndarray:
    """
    Create an adaptive grid of points for decision boundaries, balancing
    ↪ resolution and computation time.

    Parameters:
        X (np.ndarray): Input data points.
        target_points (int): Approximate number of grid points (default: 10000).

    Returns:
        np.ndarray: Grid of points shaped (n_samples, 2).
    """
    # Calculate data range with padding
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    # Handle edge cases where range is zero
    x_range = x_max - x_min
    y_range = y_max - y_min
    if x_range <= 0:
        x_range = 1e-5
        x_max = x_min + x_range
    if y_range <= 0:
        y_range = 1e-5
        y_max = y_min + y_range

    # Calculate aspect ratio and determine grid dimensions
    aspect_ratio = x_range / y_range
    nx = int(np.round(np.sqrt(target_points * aspect_ratio)))
    ny = int(np.round(np.sqrt(target_points / aspect_ratio)))
    nx, ny = max(nx, 1), max(ny, 1) # Ensure at least 1 point per axis

    # Generate evenly spaced grid
```

```

xx = np.linspace(x_min, x_max, nx)
yy = np.linspace(y_min, y_max, ny)
xx_mesh, yy_mesh = np.meshgrid(xx, yy)

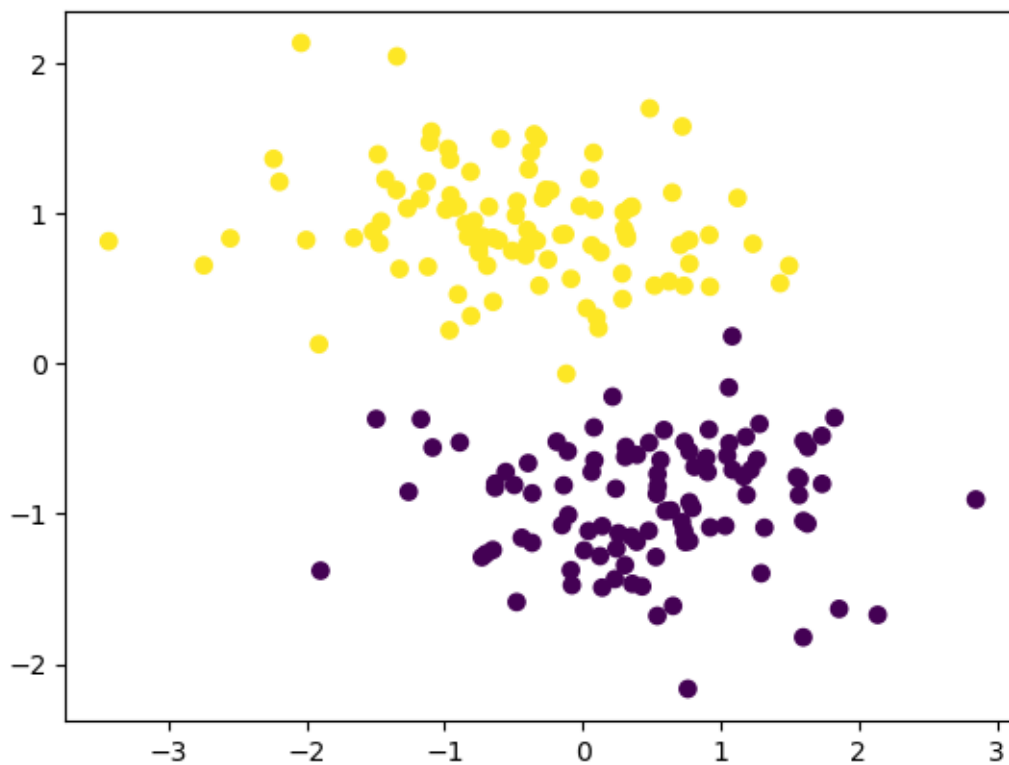
return xx_mesh, yy_mesh

X, y = make_blobs(n_samples=200, centers=2)

X = (X - X.mean(axis=0)) / X.std(axis=0)

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.show()

```



```

[318]: y = np.where(y == 0, -1, 1)

def perceptron(X, Y, max_iter=1000, learning_rate=1.0, decay_rate=0.99):
    """
    Perceptron algorithm with margin maximization.

    Parameters:
        X (np.ndarray): Input data points.
        Y (np.ndarray): Labels (-1 or 1).
    """

```

max_iter (int): Maximum number of iterations.
learning_rate (float): Initial learning rate.
decay_rate (float): Learning rate decay factor.

Returns:

tuple: Best weight vector, margin history, and weight history.
"""

```
X_augmented = np.hstack((-np.ones((X.shape[0], 1)), X))
n_samples = X.shape[0]

w = np.zeros(X_augmented.shape[1])
best_margin = -np.inf
best_w = w.copy()
margin_history = []
w_history = []

for _ in range(max_iter):
    scores = Y * np.dot(X_augmented, w)
    sorted_indices = np.argsort(scores) # Sort by ascending margin

    for i in sorted_indices:
        xi_aug = X_augmented[i]
        yi = Y[i]
        prediction = np.dot(w, xi_aug)

        if yi * prediction <= 0:
            w += learning_rate * yi * xi_aug

        # Update margin and best weights
        scores = Y * np.dot(X_augmented, w)
        min_score = np.min(scores)
        norm_w = np.linalg.norm(w)
        current_margin = min_score / norm_w if norm_w != 0 else 0

        if current_margin > best_margin:
            best_margin = current_margin
            best_w = w.copy()

    margin_history.append(current_margin)
    w_history.append(w.copy())

learning_rate *= decay_rate # Decay learning rate

# Check convergence and optimal margin
scores = Y * np.dot(X_augmented, w)
if np.all(scores > 0):
```

```

min_score = np.min(scores)
norm_w = np.linalg.norm(w)
current_margin = min_score / norm_w if norm_w != 0 else 0

# Heuristic: Check if the margin improvement is below a small threshold
margin_improvement_threshold = 1e-4
if current_margin > best_margin:
    margin_improvement = current_margin - best_margin
    if margin_improvement < margin_improvement_threshold:
        best_margin = current_margin
        best_w = w.copy()
        margin_history.append(current_margin)
        w_history.append(w.copy())
        break
else:
    break

return best_w, margin_history, w_history

# Train perceptron
w, margin_history, w_history = perceptron(X, y)

# Plot margin history
plt.plot(margin_history)
plt.xlabel('Iteration')
plt.ylabel('Margin')
plt.title('Margin History')
plt.show()

# Plot weight history
plt.plot(w_history)
plt.xlabel('Iteration')
plt.ylabel('Weight Vector')
plt.title('Weight Vector History')
plt.show()

# Create grid for decision boundary
xx_mesh, yy_mesh = make_grid(X, target_points=10000)
# Predict on grid points
X_grid = np.c_[xx_mesh.ravel(), yy_mesh.ravel()]
X_grid_augmented = np.hstack((-np.ones((X_grid.shape[0], 1)), X_grid))
p = np.dot(X_grid_augmented, w)
# Reshape predictions to match grid shape
p = np.sign(p.reshape(xx_mesh.shape))

y = np.where(y == -1, 0, 1) # Convert labels for plotting
p = np.where(p == -1, 0, 1) # Convert predictions for plotting

```

```

# Plot decision boundary
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.contourf(xx_mesh, yy_mesh, p, alpha=0.2)
plt.xlim(xx_mesh.min(), xx_mesh.max())
plt.ylim(yy_mesh.min(), yy_mesh.max())
plt.title(f'Decision Boundary with Perceptron: margin = {margin_history[-1]:.
↪2f}')
plt.show()

from sklearn.linear_model import Perceptron as SklearnPerceptron

# Train sklearn perceptron
sklearn_perceptron = SklearnPerceptron(max_iter=1000, tol=1e-3, penalty='l2',
↪alpha=0.01)
sklearn_perceptron.fit(X, y)
# Predict on grid points
p_sklearn = sklearn_perceptron.predict(X_grid)
# Reshape predictions to match grid shape
p_sklearn = p_sklearn.reshape(xx_mesh.shape)

# Compute sklearn margin
X_biased = np.hstack((-np.ones((X.shape[0], 1)), X))
scores_sklearn = y * np.dot(X_biased, np.hstack((sklearn_perceptron.intercept_,
↪sklearn_perceptron.coef_.flatten()))))
sklearn_margin = np.min(scores_sklearn) / np.linalg.norm(np.
↪hstack((sklearn_perceptron.intercept_, sklearn_perceptron.coef_.flatten()))))

# Plot decision boundary
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.contourf(xx_mesh, yy_mesh, p_sklearn, alpha=0.2)
plt.xlim(xx_mesh.min(), xx_mesh.max())
plt.ylim(yy_mesh.min(), yy_mesh.max())
plt.title(f'Decision Boundary with Sklearn Perceptron: margin = {sklearn_margin:
↪.2f}')
plt.show()

```

